SC2006 SOFTWARE ENGINEERING · TEAM 2

PathSG — System Architecture and Component Interactions

The architecture diagram in the notation of Lecture 12 (System Design), followed by the cross-component flows behind it. Derived from the codebase on 28 October 2026.

PathSG is a Singapore education-planning web app. A Vue 3 single-page app in the browser exchanges JSON over HTTP with a FastAPI service. The service keeps its data in SQLite, can rank schools through the OpenAI API, and loads its school data from CSV files that an administrator uploads. In Docker, nginx serves the site and forwards `/api` to the server, so the browser only ever talks to one origin.

The system at a glance

Tier	Runs as	Main technology	Holds or does
Browser	Vue 3 single-page app	TypeScript, Pinia, Vue Router, Tailwind	Pages, state, use-case logic, API client, session tokens in `sessionStorage`
Web server	`client` container, nginx on :8080	nginx-unprivileged	Static site, single-page fallback, `/api/*` reverse proxy, `/healthz`
Application server	`server` container, Uvicorn on :8000, one worker	Python 3.14, FastAPI, SQLAlchemy, pandas	REST API, business rules, roadmap engine, school data pipeline
Persistent data	Docker volume `pathsg-data` mounted at `/data`	SQLite, CSV files	12 tables in `main.db`; uploaded school datasets
External	HTTPS from the server; manual download by an admin	OpenAI Chat Completions; data.gov.sg	AI ranking of schools; source of the school CSVs

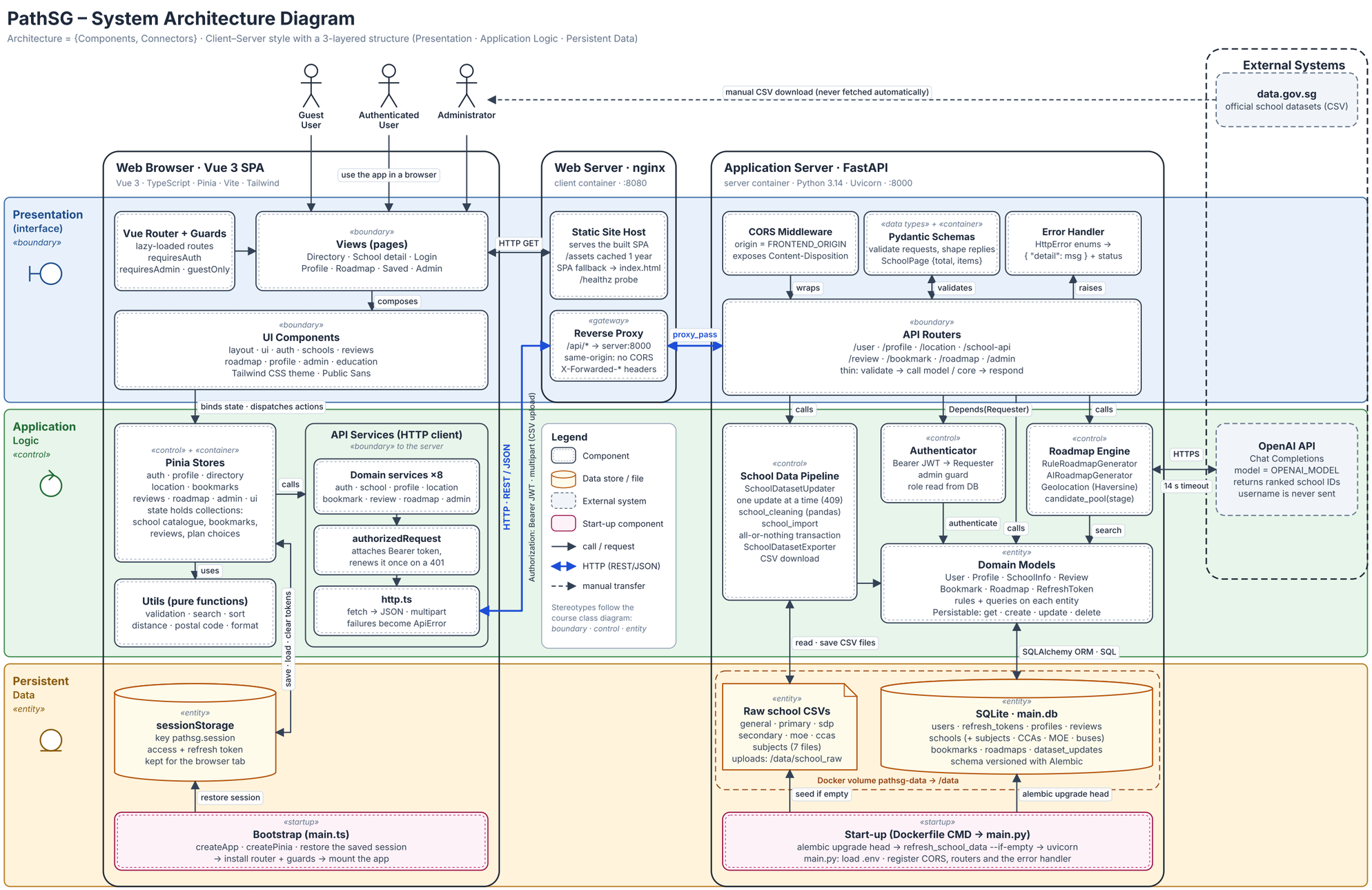
How to read the diagram

- **Components** are the boxes with a dashed inner outline; **connectors** are the arrows, labelled with the mechanism or protocol. Blue arrows are the HTTP (REST/JSON) connector between the tiers.
- The rows are the three layers: **Presentation** (`«boundary»`), **Application Logic** (`«control»`) and **Persistent Data** (`«entity»`). The columns are the tiers. Pink boxes are the start-up components.
- The strip along the bottom records how the architecture was derived: requirements, principles, patterns, constraints.

Contents

1. How the components talk: common rules
2. Sign-in and session renewal
3. Roadmap generation, including the OpenAI exchange
4. School directory and postal-code lookup
5. Administrator data update and export
6. Account features: profile, bookmarks, reviews
7. Start-up, deployment and configuration
8. Observations from reading the code

Figure 1 — PathSG system architecture diagram



Design rationale – how the architecture was derived (Lecture 12: requirements → principles → patterns → constraints)

Functional Requirements

- UC-1 Directory and school profile
- UC-2 Account, profile, bookmarks
- UC-3 Rule-based and AI roadmaps
- UC-4 School reviews
- UC-5 Admin data update and export

Non-functional Requirements

- PERF search, profile ≤ 2 s · AI plan ≤ 15 s
- SEC bcrypt · 5-min JWT + refresh token
- QUAL modular routers and components
- REL all-or-nothing data refresh · health checks
- PORT Docker images for amd64 and arm64

Design Principles

- Separation of concerns: boundary/control/entity
- Modularity: router + store + service per feature
- Low coupling: view → store → service, one way
- Single responsibility: thin routers, rules in models
- Reuse: shared UI components, HttpError enums

Design Patterns

- Client–Server: SPA ⇄ REST API
- Layered: presentation · logic · data
- Reverse Proxy: nginx, same-origin /api
- Strategy: rule-based vs AI roadmap generator
- Facade: API services · Dependency Injection

Constraints

- Languages: TypeScript, Python 3.14
- Libraries: Vue 3, Pinia · FastAPI, SQLAlchemy
- Communications: HTTP(S), REST/JSON, JWT
- Data: SQLite, a single server instance
- External: OpenAI API · no live map API

1 How the components talk: common rules

1.1 The path of an API request

1. A view calls a store action. The store calls an API service (`schoolService`, `roadmapService`, ...), which calls `http.ts`. The URL is `VITE_API_URL` plus the path; the Docker image builds with `/api`, so requests go to the page's own origin.
2. nginx matches `location /api/` and `proxy_pass`es to `${API_UPSTREAM}/` (default `http://server:8000`). The trailing slash removes the `/api` prefix. nginx adds `Host`, `X-Real-IP`, `X-Forwarded-For` and `X-Forwarded-Proto`.
3. FastAPI runs the CORS middleware, finds the router, resolves dependencies (`Requester` on protected routes) and lets Pydantic validate the body or query.
4. The route, a plain `def` that FastAPI runs in a worker thread, calls a model or a `core/` class. Each model call opens its own short SQLAlchemy session (the `Persistable` mixin). Only the school data import holds one session across a whole transaction.
5. A response schema shapes the JSON. Failures are `HttpError` enum members that `main.py` turns into `{"detail": "..."}` with the status code.

> **Development mode.** `npm run dev` serves the client on :8080 and calls the API directly at `http://127.0.0.1:8000` (`VITE_API_URL`). That is a cross-origin call, which is why the server has CORS middleware: it allows only `FRONTEND_ORIGIN`, with credentials, and exposes `Content-Disposition` so the client can read export file names.

1.2 Layering rules the code follows

- Calls go one way: view, then store, then service, then the network. Stores never import views; services never import stores.
- On the server, routers stay thin: validate through a schema, call a model or `core/` class, shape the response. Rules and queries live in `models/` and `core/`.
- Utilities (`client/src/utils`) are pure functions with no state, so validation and distance rules can be tested alone.

1.3 Errors

Status	Raised by	What the browser does
401	`Requester` (missing, bad or expired JWT); refresh and login failures	`authorizedRequest` renews the tokens once and retries; a refused renewal ends the session
403	Admin guard, owner checks	Shows the message
404, 409, 422	Models and `core/` (not found, duplicate, invalid input)	Shows the message beside the form or in the banner
503, 504	Database unreachable; OpenAI failure or timeout	A popup (admin pages) or a failed plan state (AI roadmap)
no status	Server unreachable, request aborted, or a non-JSON reply (for example from nginx)	`ApiError(0)` with the generic text "Unable to complete your request at this time. Please try again later."

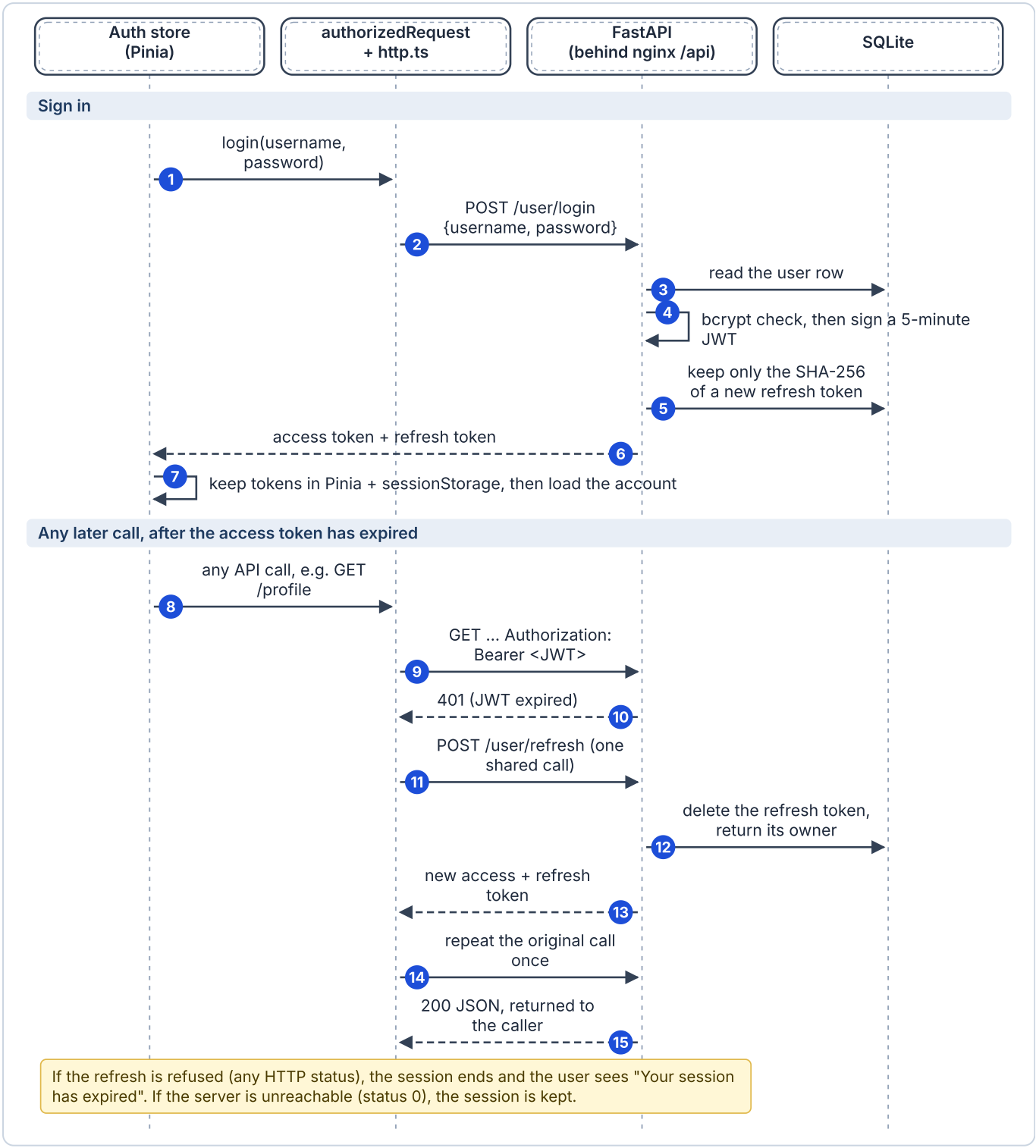
1.4 Limits and timeouts that tie components together

Limit	Value	Where it is set
Access token lifetime	5 minutes	`models/user.py` (HS256 JWT)
Refresh token lifetime	7 days, single use	`models/refresh_token.py` (SHA-256 digest stored)
AI plan: browser gives up	15 s	`roadmapService.ts` (`AI_TIMEOUT_MS`)
AI plan: OpenAI call gives up	14 s, no retries	`core/roadmap_ai.py` (`AI_TIMEOUT_SECONDS`)
Roadmap size; saved roadmaps per account	1 to 3 schools; one	`PLAN_LENGTH` ; unique `user_id`
Directory page size; candidate pool	400 per request; up to 1000	`schoolService.ts` ; `core/roadmap_pool.py`
CSV upload size	20 MB per file	`SCHOOL_MAX_UPLOAD_MB` in `school_import.py`
Concurrent school data updates	one	In-process lock in `dataset_update.py`

2 Sign-in and session renewal

Components involved: Auth store, `authService`, `authorizedRequest`, `http.ts`, `sessionStorage`, the user router, `Authenticator`, the `User` and `RefreshToken` models, SQLite.

Figure S1 — Sign-in, an authorised call, and single-flight token renewal



Step	What happens
1	`auth.login()` calls `authService.login()`, which uses `http.ts` (no token yet).
2	JSON request through nginx. `Credentials` validates the body.
3	`User.login()` loads the account by username (401 if unknown).

Step	What happens
4	`bcrypt.checkpw` against the stored hash (401 if wrong). The access token is an HS256 JWT whose `sub` is the user id, signed with `SECRET_KEY`.
5	`RefreshToken.issue()` stores the digest with a 7-day expiry and purges expired rows. The token itself is never stored.
6	The token pair returns to the browser.
7	`GET /user/identity` gives username and role. Then profile, bookmarks and the saved roadmap load in parallel, and the saved postal code becomes the directory's distance origin.
8	Services call `authorizedRequest`, which takes the access token from the signed-in session. A signed-out caller gets 401 locally.
9	Every protected call carries the header.
10	`Requester` rejects the token. `User.authenticate()` decodes the JWT and reads the user row on every request.
11	The first 401 starts the renewal; other 401s that arrive meanwhile wait for the same promise, because a refresh token works only once.
12	`DELETE ... RETURNING` in one statement, so even racing requests cannot reuse a token.
13	A fresh pair, or 401 if the token is unknown, used or expired.
14	The new tokens are saved to Pinia and sessionStorage, then the original request is retried a single time.
15	The caller never sees the expired token.

Where each rule is enforced

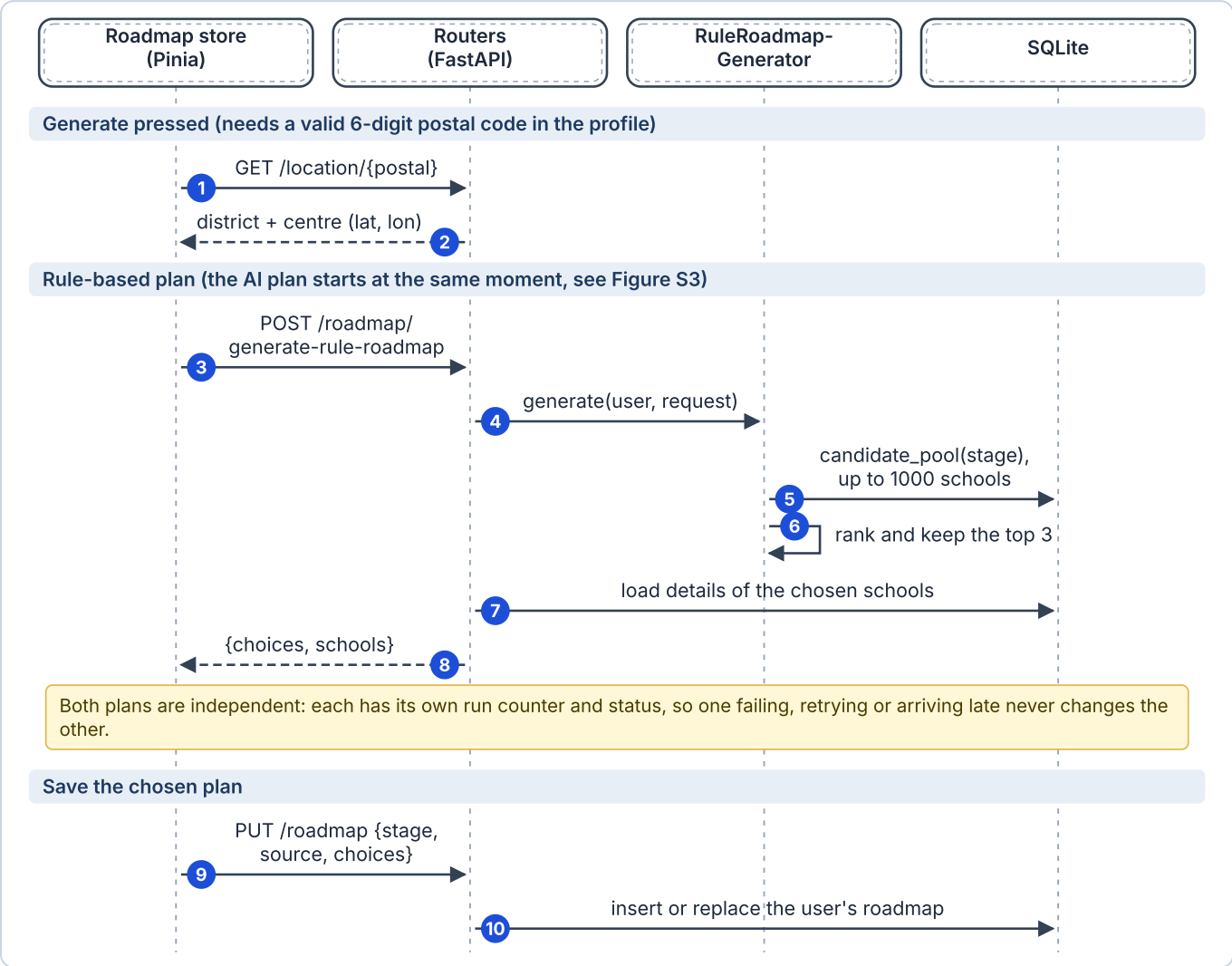
- **Page guards** (`requiresAuth`, `requiresAdmin`, `guestOnly`) only decide which page to show. The server enforces access: `Requester` on every protected route, and `get_admin_requester` on the whole `/admin` router.
- The role is read from the database on each request, so promoting or demoting an account takes effect without a new token.
- **Page reload:** `main.ts` calls `auth.restore()` before installing the router. It reads `sessionStorage`, calls `/user/identity` (renewing once on 401), so reloading a protected page does not bounce to the login page. Tokens last for the browser tab.
- **Log out** clears the stores and storage first, then asks `POST /user/logout` to forget the refresh token; a failure of that call is ignored.

3 Roadmap generation, including the OpenAI exchange

3.1 Two independent plans

Pressing generate starts the postal-code lookup, the rule-based plan and the AI plan together. Each plan has its own request, status and run counter in the Roadmap store. A plan that fails can be retried without touching the other, and a response from a superseded run is discarded.

Figure S2 — Generating and saving a roadmap



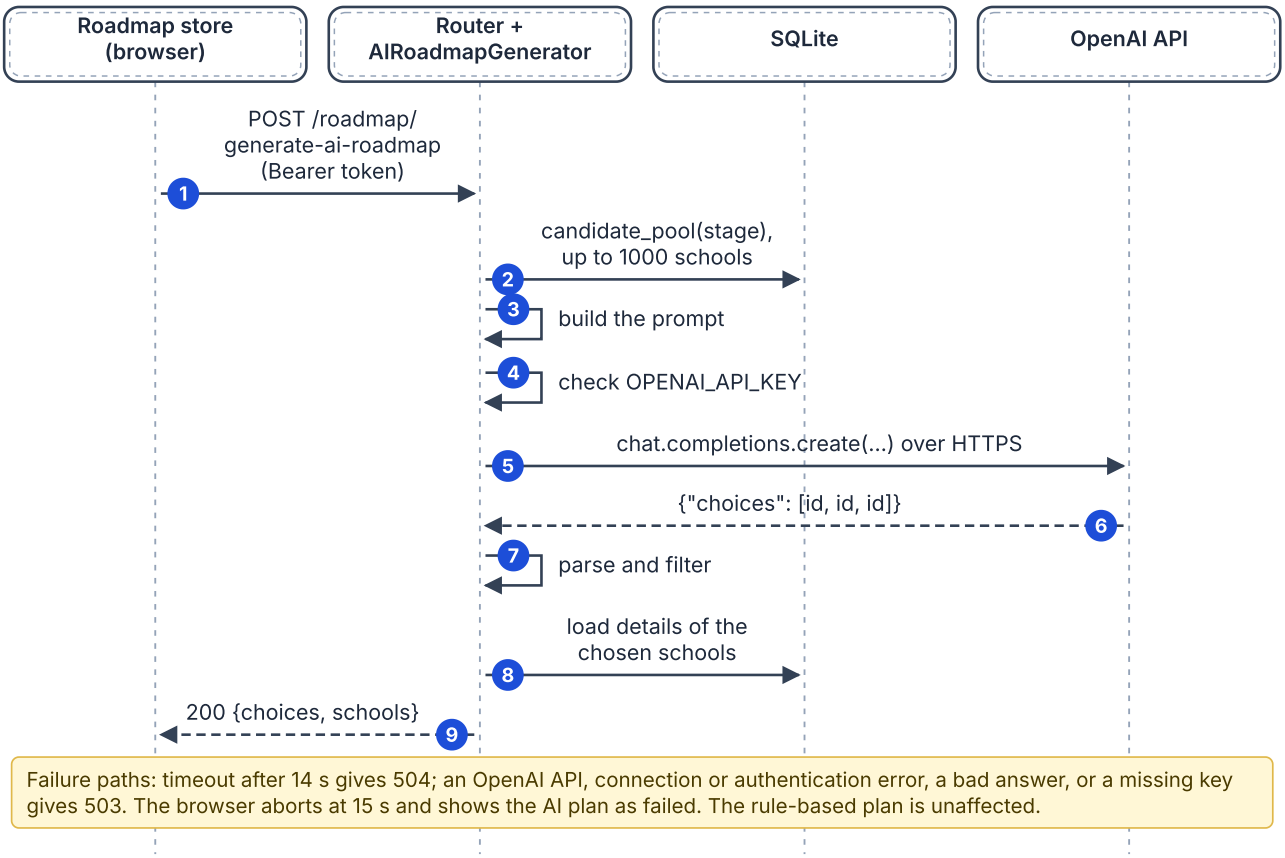
Step	What happens
1	`location.locate()`, cached per code. The server maps the first two digits to a postal district and answers with that district's centre.
2	404 if the code cannot be placed. The browser measures distances from this point with the same Haversine formula the server uses.
3	Body: `stage`, `postal`, `previous_academic_score`. The score is sent only for the stage the profile targets.
4	The router hands the request to `RuleRoadmapGenerator`.
5	`SchoolInfo.search(stage=..., limit=1000, load_programmes=True)`. Integrated schools appear in every stage they cover.
6	Primary: by distance from the postal district centre. Secondary: schools the score reaches (against the 2017 PSLE cut-off) first, then by distance. Post-secondary: by number of subjects, then CCAs. University and preschool: no rule-based plan.

Step	What happens
7	The router builds `{choices, schools}`; an id with no school is left out.
8	The store remembers the schools, shows the plan as ready, and ignores the response if a newer run has started.
9	`source` is "Rule-Based Plan" or "AI-Generated Plan". `Roadmap` checks the stage, the source and that there are 1 to 3 integer ids.
10	One roadmap per account (unique `user_id`). The response lists the schools' details so the page can render it.

3.2 How the OpenAI API is used

The AI plan is produced by `AIRoadmapGenerator` in `core/roadmap_ai.py`. It is the only place the server calls an external service at request time.

Figure S3 — The AI roadmap request and the OpenAI exchange



Step	What happens
1	`generateAiPlan()` starts a 15-second `AbortController`. The route is a plain `def`, so FastAPI runs it in a worker thread and the wait does not block other requests.
2	Same query as the rule-based plan. If the pool is empty the route returns an empty plan and never calls OpenAI.
3	Target stage, previous score (or "not provided"), postal code (or "not provided"), a note on how to read the score for that stage, the candidates as compact JSON, and numbered criteria (only listed ids, at most 3, distance only where it applies). The username is never included.
4	An empty key (compose passes an empty string when unset) answers 503 at once instead of making a call that would fail with 401.
5	`OpenAI(timeout=14, max_retries=0)`. Model is `OPENAI_MODEL` (default `gpt-5.5`), one user message, `reasoning_effort="low"`, and a strict JSON schema `{"choices": [integer]}`.

Step	What happens
6	`choices[0].message.content` is the JSON text.
7	Keep only ids that are in the candidate pool, drop duplicates, take the first 3. Unparsable JSON or no valid id gives 503.
8	`schools_of()` returns the schools in the chosen order.
9	The store marks the AI plan ready. The user can save it (source "AI-Generated Plan") or retry it on its own.

What leaves the system

Sent to OpenAI	Never sent
Target stage, and the previous academic score if the student entered one	Username, user id, password, tokens
Postal code, if provided	Reviews, bookmarks, the saved roadmap
For each candidate school: id, name, level, type, zone, postal code, 2017 cut-off, and programme flags (SAP, autonomous, gifted, IP, IB)	School address, programme titles, gender mix, 2017 rank (left out to keep the prompt short and the answer fast)
Fixed instructions and the answer format	

What happens when it goes wrong

Condition	Server answer	In the browser
`OPENAI_API_KEY` empty	503 at once, no network call	The AI plan is shown as failed. The rule-based plan is not affected, and the AI plan can be retried on its own.
OpenAI API, connection or authentication error	503 "Unable to generate AI recommendation at this time. Please try again."	
No answer within 14 s	504 "The AI recommendation took too long to generate. Please try again."	
Answer is not valid JSON, or has no valid id	503 (same message as above)	
Browser aborts at 15 s, or the server is unreachable	none	
No schools loaded for the stage	200 with an empty plan; OpenAI is not called	An empty plan

Time budget. The server stops waiting at 14 s so that its answer, with the proxy and network time, still reaches the browser before the browser's 15 s limit (SRS PERF-7). nginx's default 60 s proxy timeout is never the limit. **Configuration:** `OPENAI_API_KEY` and `OPENAI_MODEL` are read from the environment; compose passes both to the server container. Without a key the rest of the system works normally.

3.3 The saved roadmap

Call	Effect
`GET /roadmap`	The saved roadmap with its schools; 404 when none, which the browser treats as "no saved roadmap".
`PUT /roadmap`	Saves, replacing any existing one. The model checks the stage, the source ("Rule-Based Plan" or "AI-Generated Plan") and 1 to 3 integer ids.
`PATCH /roadmap`	Changes only the school choices. To offer swaps, the browser first loads the whole stage through the directory store.
`DELETE /roadmap`	Removes it (204); a repeat call does nothing.

An id whose school no longer exists, for example after a data update, is left out of the response rather than failing it.

4 School directory and postal-code lookup

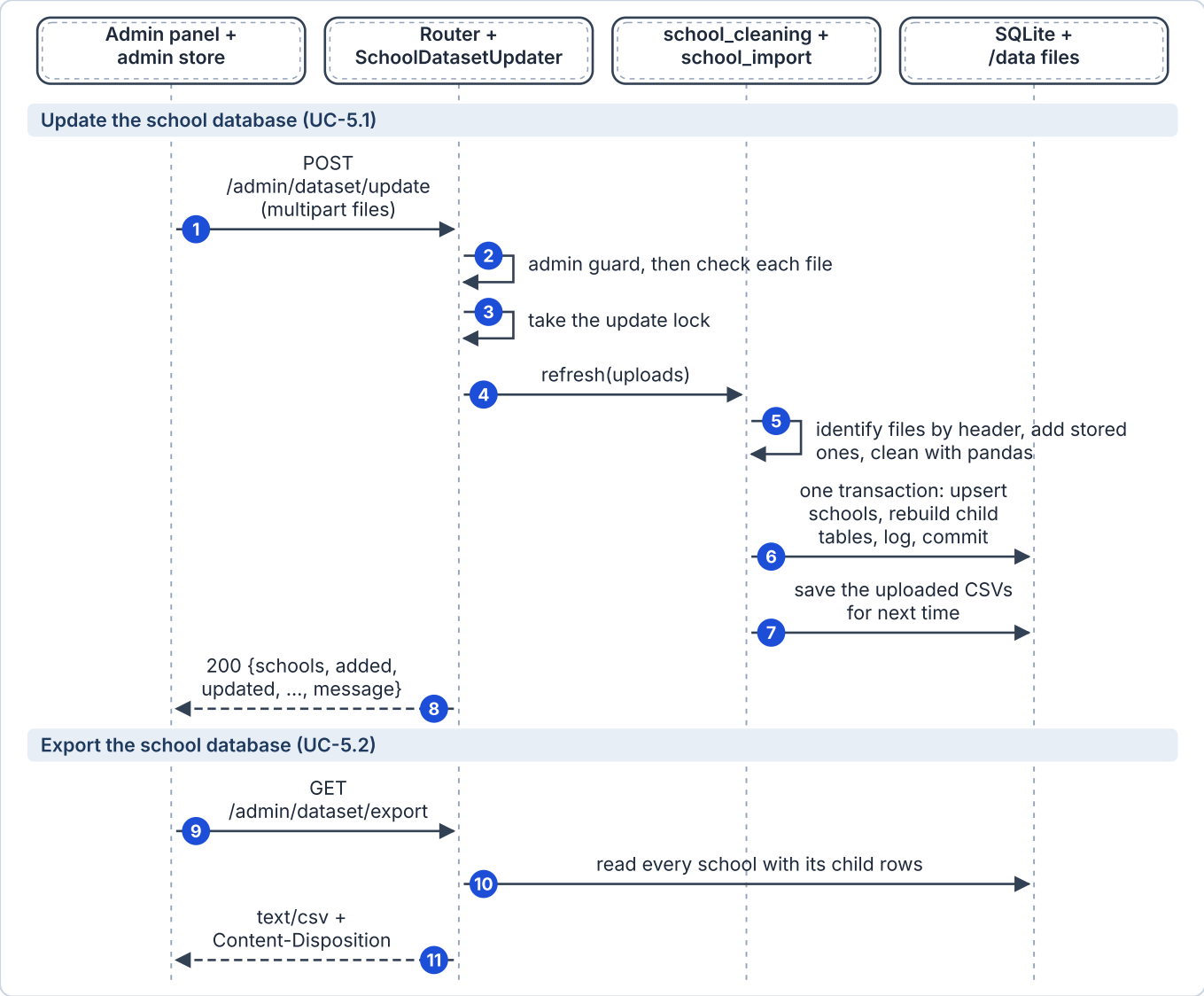
1. **Loading a tier.** The Directory view asks the directory store for a tier (`primary`, `secondary`, `postsec`, `uni`). `fetchSchoolsOfTier()` pages through `GET /school-api?stage=...&limit=400&offset=...` until the total is reached. The list is cached per tier, and concurrent requests for the same tier share one fetch.
2. **Searching and filtering** then run in the browser over that list (`schoolSearch`, `searchQuery`): keyword, school type, distance, sort order and pagination do not call the server again.
3. **Distance.** The reference postal code is the profile's saved code after sign-in, otherwise a default. `location.locate()` asks `GET /location/{postal}` once per code; an unreachable server is not cached so the next call retries, while a 404 is remembered as "cannot be placed". The browser then measures Haversine distance from the district centre to each school's coordinates, the same point and formula the server's rule-based plan uses.
4. **School profile.** `GET /school-api/{school_key}` returns the school with its subjects, CCAs and MOE programmes (404 for an unknown key, which the browser treats as "no such school"). Reviews load separately from `GET /review/school/{school_id}`.
5. **Freshness.** After an administrator updates the data, the admin store clears the cached catalogue so the next visit refetches it.

Public endpoints (no token needed): `/school-api`, `/school-api/options`, `/school-api/{school_key}`, `/location/{postal}` and `GET /review/school/{school_id}`. Guests can browse everything except roadmaps, profile, bookmarks and the admin panel.

5 Administrator data update and export

An administrator downloads the school datasets from data.gov.sg and uploads them; the system never fetches them itself. The same cleaning and loading code runs at start-up (`scripts.refresh_school_data`) and from the admin panel.

Figure S4 — Updating and exporting the school database



Step	What happens
1	The admin picks one or more CSVs. `adminService.updateDataset()` sends a `FormData` with a repeated `files` field and the Bearer token.
2	`get_admin_requester` guards the whole `/admin` router (role read from the database; 403 otherwise). Every file must be a non-empty `.csv` within `SCHOOL_MAX_UPLOAD_MB` (422 otherwise).
3	A non-blocking in-process `threading.Lock`: a second update gets 409 while one runs. This is why the server runs one worker.
4	Cleaning and loading is shared with `scripts.refresh_school_data`.
5	Each upload is recognised by its column headers, so file names do not matter. Kinds not uploaded come from the last upload (`SCHOOL_RAW_DIR`) or the bundled copy. All seven kinds are needed. Cleaning touches no database; bad data raises before any write (422).

Step	What happens
6	Schools are matched by `school_key`; subjects, CCAs, MOE programmes and buses are rebuilt; a `dataset_updates` row is added in the same transaction. Any error rolls everything back. Database unreachable gives 503.
7	Only after the commit. If saving fails the update still succeeds, with a warning in the summary.
8	The store shows the success banner, clears the cached school catalogue so the directory refetches, and reloads `GET /admin/dataset` (record count, last update).
9	Same admin guard. Nothing about accounts, reviews, bookmarks or roadmaps is read.
10	`SchoolInfo.all_with_details()`. An unreachable database or no schools gives 503 "Export Failed ...".
11	File name `pathsg_schools_YYYYMMDD.csv` (Singapore date). The browser stores it through a temporary object URL; any failure shows one popup and saves nothing.

Outcome	Status	What the admin sees
No file picked, or a file that is not a non-empty `.csv` within the limit	422	Message beside the form
Another update is running	409	Message beside the form
Data cannot be processed (unrecognised columns, two files of one kind, bad values)	422	"Database Update Failed: ..." popup; nothing changed
Database unreachable	503	"Database Update Failed ..." popup; nothing changed
Success	200	Banner plus the counts: schools, added, updated, `not_in_files` (kept), and any warnings

6 Account features: profile, bookmarks, reviews

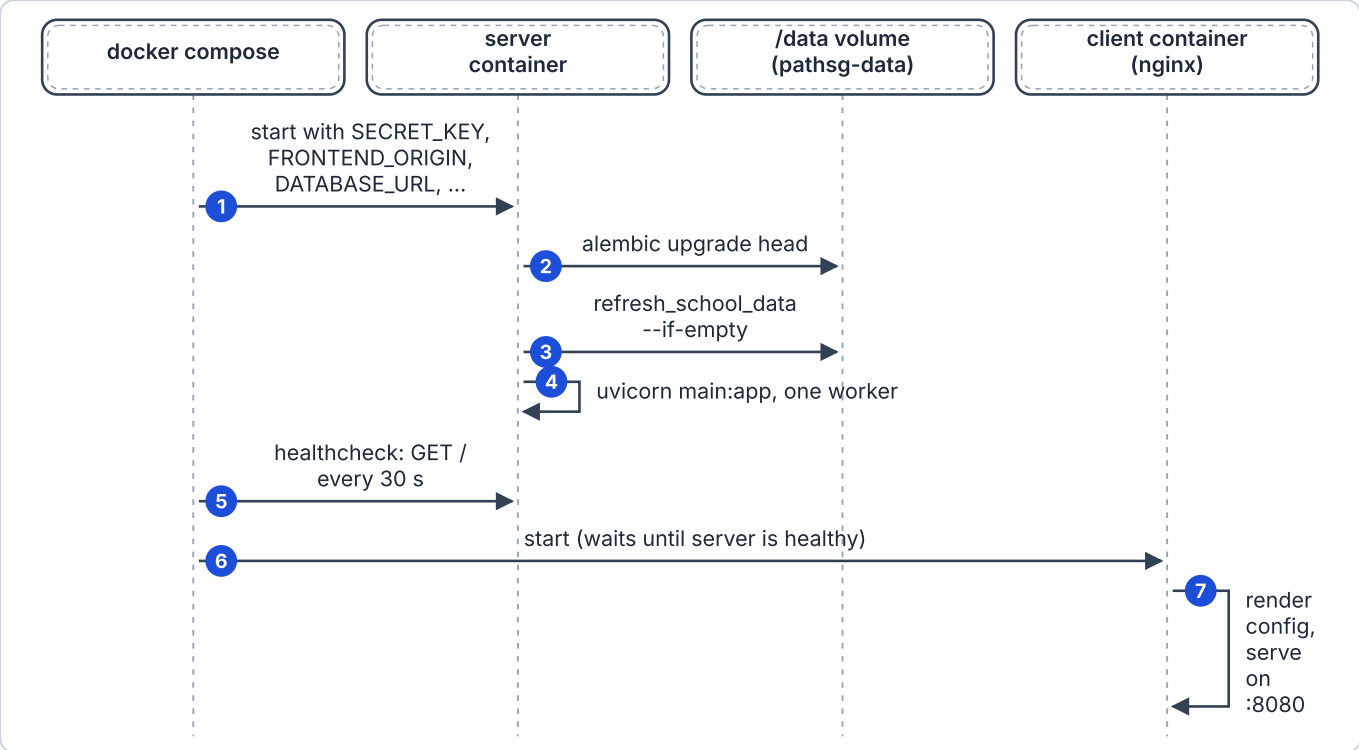
Feature	Endpoints	Server rules	Browser behaviour
Profile	`GET /profile` `PUT /profile`	Postal code must be 6 digits; scores must fit the target level (for example PSLE AL 4 to 32). One row per user, replaced on save. A new account gets starting values. No name or NRIC is stored.	The form checks the same rules first (`profileValidation`). The profile loads at sign-in with bookmarks and roadmap, and its postal code becomes the distance origin.

Bookmarks	`GET /bookmark`
	`PUT /bookmark/{id}`
	`DELETE /bookmark/{id}`
		Unique per user and school; add and remove are idempotent (204); the school must exist (404).	The store marks that school as pending so it cannot be double-clicked. After an add it reloads the list, because the server holds the school details; after a remove it drops the school locally.

Reviews	`GET /review/school/{id}`
	`POST` same path
	`PATCH` and `DELETE /review/{id}`
		One review per user per school (409). Only the author may edit or delete (403). HTML tags are stripped, the text is trimmed, blank text is refused, and there is a word limit. Reads join the author's username, newest first.	The store keeps reviews per school and replaces the changed review with the server's copy.

7 Start-up, deployment and configuration

Figure S5 — Container start-up order



Step	What happens
1	`SECRET_KEY` must be set or compose refuses to start. The image runs as an unprivileged user.
2	Creates or upgrades the 12 tables in `/data/main.db`.
3	Loads the bundled CSVs only when no schools exist, so a new deployment starts with data.
4	`main.py` loads `.env`, adds CORS for `FRONTEND_ORIGIN`, includes the eight routers and installs the `HttpError` handler.
5	Starts with a 10 s grace period; the server answers a small JSON message when ready.
6	`depends_on: server: condition: service_healthy`.
7	The nginx template is rendered with `API_UPSTREAM` (default `http://server:8000`). It serves the built site, proxies `/api/`, and answers `/healthz`.

When the page opens, the browser-side start-up runs in `main.ts`: create the app and Pinia, restore the saved session (section 2), install the router and its guards, then mount.

Settings that connect the components

Setting	Default	Used by
`SECRET_KEY`	required	Server: signs access tokens
`FRONTEND_ORIGIN`	required (compose: `PUBLIC_ORIGIN`, default `http://localhost:8080`)	Server: the one origin CORS allows
`DATABASE_URL`	`sqlite:///./main.db` (Docker: `sqlite:////data/main.db`)	Server and Alembic

Setting	Default	Used by
`OPENAI_API_KEY`, `OPENAI_MODEL`	none; `gpt-5.5`	Server: AI roadmap
`ADMIN_USERNAMES`	empty	Server: extra admins, honoured only by `POST /school-api/admin/refresh`
`SCHOOL_RAW_DIR`, `SCHOOL_MAX_UPLOAD_MB`	`data/raw` (Docker: `/data/school_raw`); 20	Server: stored CSVs; upload limit
`VITE_API_URL`	build argument `/api`	Client build: base URL of API calls
`API_UPSTREAM`	`http://server:8000`	nginx: where `/api/` goes

Build and release

Pull requests run `ci.yml`: the client (format, type-check, Vitest, build), the server (Ruff lint and format, pytest), a build of both Docker images, and a validation of `docker-compose.yml`. Pushes to `main` and tags such as `v1.2.3` run `publish.yml`, which repeats CI and then pushes `linux/amd64` and `linux/arm64` images of the client and the server to GitHub Container Registry. A host updates with `docker compose pull && docker compose up -d`.

8 Observations from reading the code

These came up while tracing the flows above. They are findings to consider, not changes made.

> **1. Uploads through nginx may hit its default body limit.** `nginx.conf.template` does not set `client_max_body_size`, so nginx's documented default of 1 MB applies to `/api` requests, while the server accepts 20 MB per file. The seven bundled CSVs total about 1.4 MB (the largest, `subjects.csv`, is 0.68 MB), so uploading all of them in one request would exceed 1 MB; single files fit. nginx would answer 413 with an HTML body, which the client shows as the generic error. Adding `client_max_body_size 25m;` to `location /api/` would align the two limits. This was read from the configuration, not tested end to end.

1. **Session design differs from the SRS.** SEC-5 describes JWTs in HTTP-only cookies for 24 hours. The code uses a 5-minute Bearer access token and a single-use 7-day refresh token kept in `sessionStorage`. The SRS section on communications already describes the code's behaviour.
2. **No database backup.** SAFE-1 asks for a daily backup of the SQLite file. No backup job exists in the repository or the compose file; the `pathsg-data` volume is the only copy.
3. **One server instance only.** SQLite and the in-memory update lock mean the server must run one process and one replica (the Dockerfile enforces one worker).
4. **Two ways to be an administrator.** `/admin/*` and `/user/identity` use the stored role, while `POST /school-api/admin/refresh` also honours `ADMIN_USERNAMES`. An account listed only there cannot use the admin panel's own routes.